

VAHAN SYSTEM OVERVIEW REPORT

Data collection, administration, and planned expansion

Updated for the current system - 10 October 2026 (UTC+07:00).

This report explains how VAHAN reports are collected, validated, saved and shared. It retains the original section order (1-10 and 2.1) and describes the current system components. English diagrams appear in their corresponding sections alongside the explanation and tables.

Review basis: the current local source and repository documentation. Features described as current are present in this checkout; this document update does not certify a deployed version, a live collection run or measured throughput.

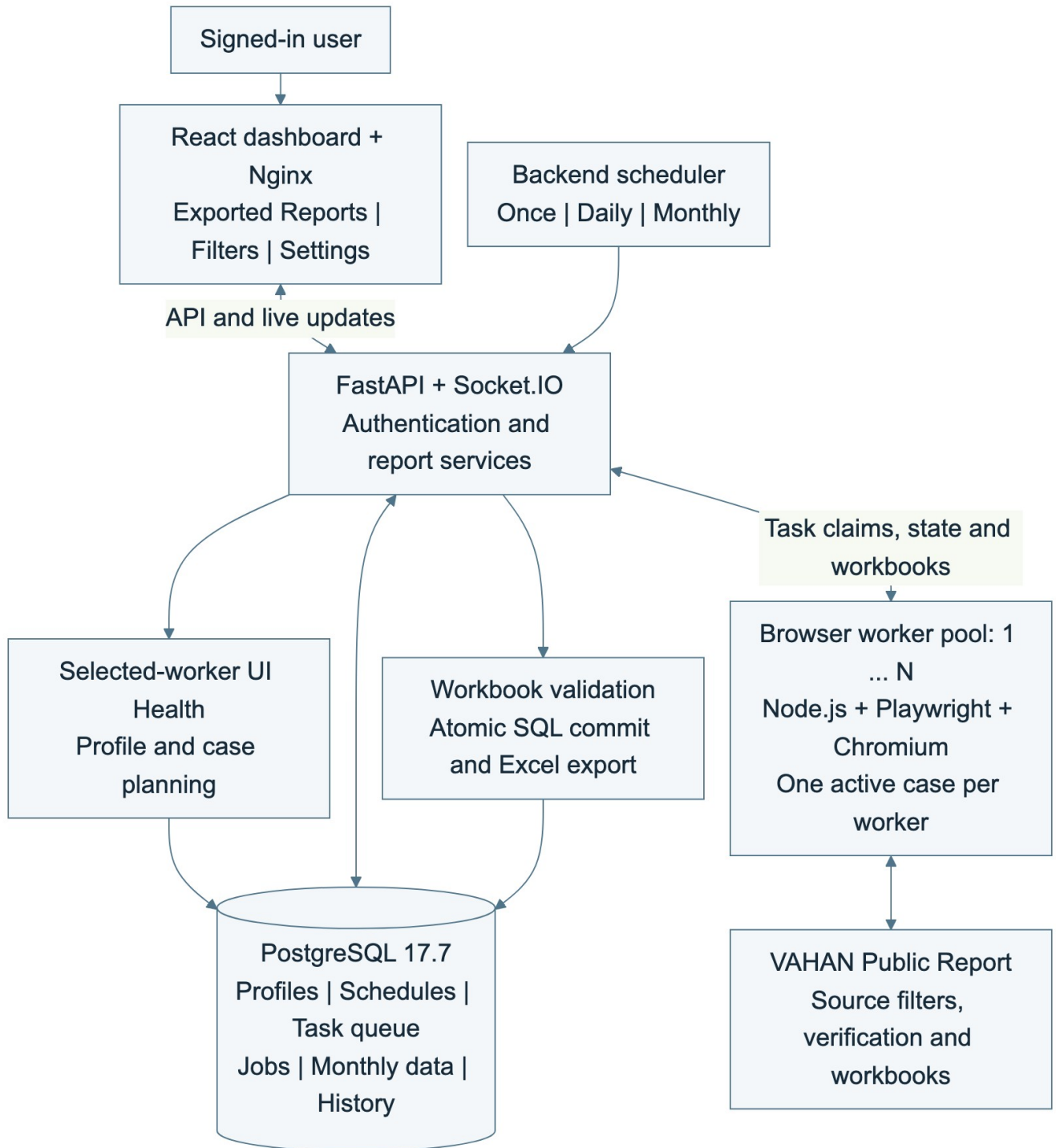
Overall architecture

The current system uses a React/TypeScript administration dashboard served by Nginx, a FastAPI service with Socket.IO and a backend scheduler, PostgreSQL for durable records and task claims, and a configurable pool of Node.js/Playwright Chromium workers. Figure 1 shows this architecture.

Component	Current responsibility
Dashboard + Nginx	Exported Reports, Filters and Settings; authenticated API requests and live progress updates.
API + scheduler	Authentication, profile planning, time-based starts, UI Health, queue coordination, workbook ingestion and export.
PostgreSQL 17.7	Accounts, profiles, schedules, queue cases, jobs, errors, monthly report data and persistent browser state.
Browser workers	One active case per worker; claim work through the API, fill and verify VAHAN filters, obtain source workbooks and report results.

The Compose stack runs on one host. Workers claim PostgreSQL task rows in SQL transactions; there is no Redis service or separate shared CAPTCHA service in the current Compose configuration. Browser-worker replicas and the API concurrency limit are separate settings.

Figure 1. Current system architecture



Current single-host Compose architecture. The PostgreSQL task queue is part of the database; workers claim tasks through the API. No Redis or separate CAPTCHA service is deployed.

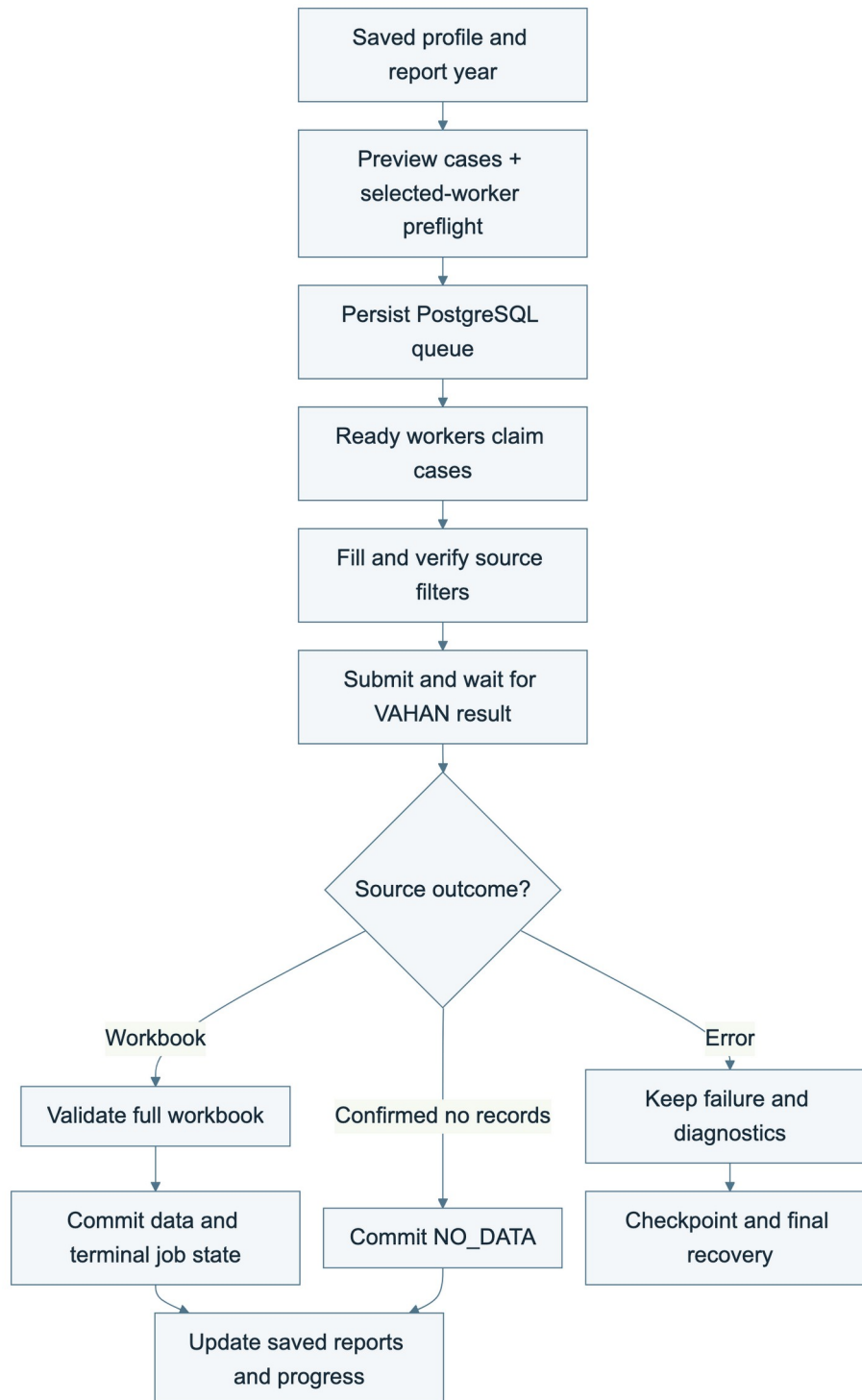
1. Data collection process

A saved filter profile is expanded into valid cases using current VAHAN options. Selected workers pass UI Health before new work is allowed. The full workbook or a confirmed no-data result is committed before a case becomes successful. Figure 2 shows the outcomes.

Step	How the system works	User outcome
Define scope	Use a saved profile/revision, report year and valid State/RTO/filter combinations; preview the plan, up to 3,000 cases.	The run has a reproducible scope.
Collect data	Ready workers claim cases, open VAHAN, fill dependent controls and recheck observed values.	Different cases can run concurrently within the selected limit.
Import data	Download the complete workbook; validate sheets, reporting context and values; save through the API.	Saved data becomes available in Exported Reports.
No data or error	Save NO_DATA only after source confirmation; keep timeouts, verification and parsing failures as errors.	Empty results remain distinct from execution failures.

OTHERS retains its source label. A reported zero stays zero; an absent monthly value remains blank. A worker waits for the save acknowledgement before releasing the case.

Figure 2. Current collection flow



A successful case requires committed workbook data or committed, explicitly confirmed no data.

2. Administration interface functions

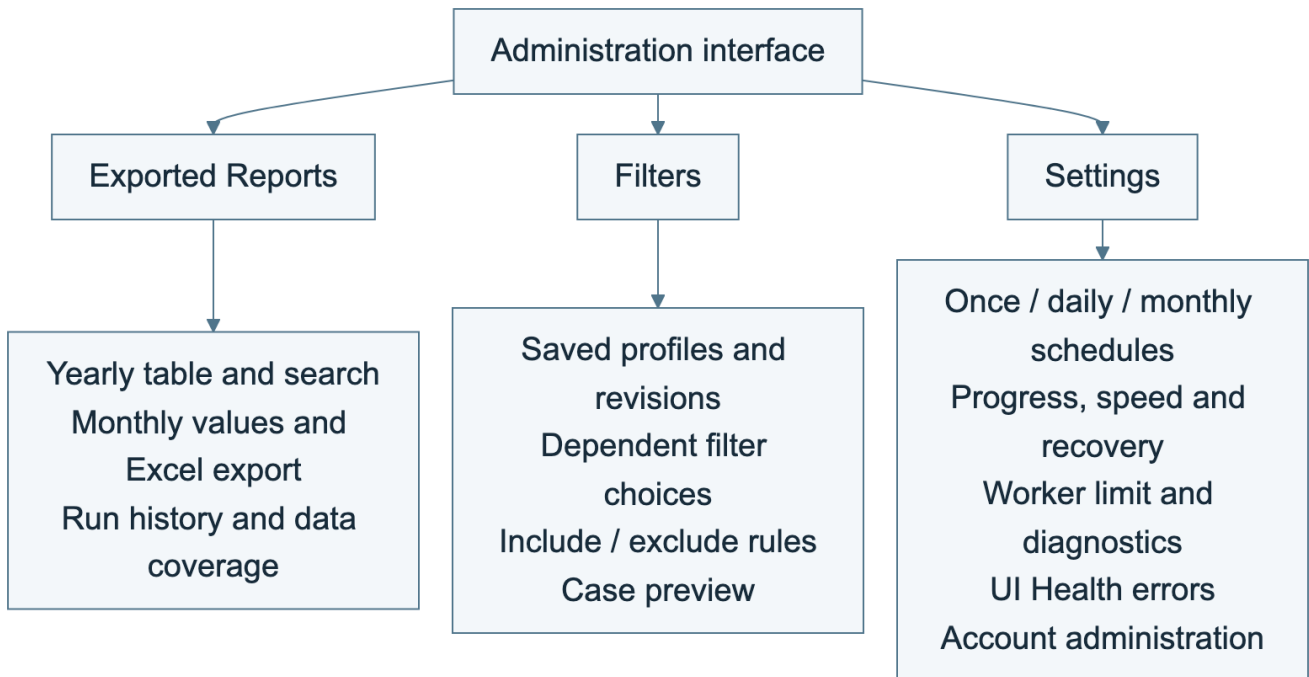
The navigation keeps three main areas: Exported Reports on the left, Filters in the middle and Settings on the right. Signed-in members can view and export shared reports; administrators also manage profiles, schedules and accounts. Figure 3 summarizes the functions.

Function	Actions and information shown	User benefit
Configure filters	Create named profiles with revisions; choose reporting year and fields; set fixed/iterated values, inclusion/exclusion and combination rules; preview valid cases.	Reuse a consistent reporting scope.
Create and run reports	Use a saved profile in Settings; add a future one-time, daily or monthly schedule; set time zone and parallel-task limit.	Start runs from a persisted plan.
Monitor progress	View processed/total cases, state, cases/min, System activity, worker observations and copyable diagnostics.	See progress and delays requiring attention.
Handle errors	Retry failures at each checkpoint (at least 10 cases, sized for concurrency), then one final recovery pass; list unresolved cases and attempt history.	Recover without repeating saved successes.
Run history	Review session/case status, timestamps, errors and files; delete eligible sessions from history and restore deleted sessions.	Trace previous runs while retaining saved report data.
Exported reports	Choose dataset/year, search State/RTO, page through Maker/month data and export the matching saved rows.	Reuse and share the collected dataset.
Data coverage	Compare the known plan with confirmed saved results; show collected/missing offices and Last data saved in GMT+7.	Identify gaps; coverage does not automatically collect them.
Schedule report runs	Once, Every day or Every month; default Vietnam time, optional India time; pause/continue, adjust the limit while paused, enable/disable repeats or delete eligible schedules.	Control timing and remaining work.
Account administration	View identity/role, log out and change password; admins create, enable/disable, reset and assign roles to accounts.	Control access and responsibilities.
Source-page checks	UI Health preflight for selected workers before planning, Maker loading and new queue execution; no separate source-check schedule is offered in navigation.	Detect incompatible page changes before work.
DOM error reporting	Show codes, affected controls, expected/observed structure, worker evidence and copyable diagnostics; block new assignments when invalid.	Support repairs and prevent incorrect automation.

Settings uses compact run cards. Show form opens the schedule dialog; Hide form closes it. Account and diagnostic details use bounded dialogs. The separate Create Report/manual-run and standalone UI Health schedule pages are no longer part of the current navigation.

Timing limitation: current run cards render progress and processing speed. Timing logic exists, but remaining time and estimated finish are not rendered by the current RunScheduleProgress component. These must not be presented as delivered UI features.

Figure 3. Administration interface functions



The hierarchy preserves Exported Reports on the left, Filters in the middle and Settings on the right. The main branches have two downward levels.

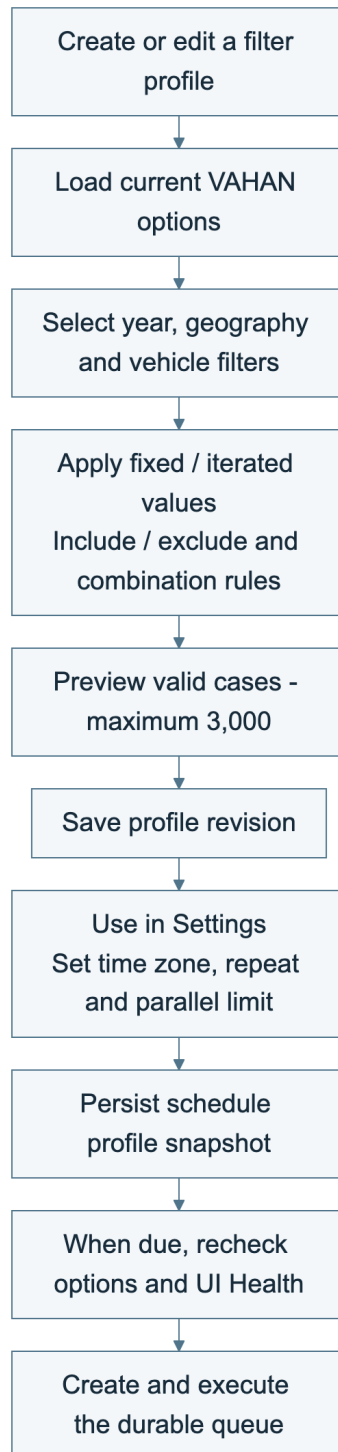
2.1. Edit filter configurations and select cases

Editable filter profiles are now present. The system is no longer limited to an uneditable current-year Two Wheeler/electric State-RTO matrix. The supported report layout remains Calendar Year / Maker / Month Wise, producing 12 monthly columns for one reporting year. Figure 4 shows profile planning and selection.

Configuration	Current behavior
Period and year	Choose the reporting year; the source schema limits it to the current or an earlier calendar year. The supported axes remain Maker / Month Wise.
Geography	Choose Delhi NCR region, State and RTO; dependent options are loaded and validated against VAHAN.
Vehicle attributes	Configure group, subcategory/class, EV/fuel, active/archive, emission, Maker, status, owner type, vehicle type and fitness where supported by the profile.
Selection rules	Fix a value or iterate supported choices; apply include/exclude and combination rules; preview the resulting cases with a 3,000-case ceiling.
Saved revision	Save the profile and select it in Settings. A schedule captures its profile revision and year; later edits do not silently modify that schedule.
Case scope	A restricted profile can represent one office or selected offices. Current execution uses schedules; the old run-all/start-from-RTO/manual-run screen is not the current interface.

Changing a parent filter requires dependent options to be checked again. The worker verifies expected and observed values before Apply. To use a changed profile revision, create a new schedule rather than editing an active run indirectly.

Figure 4. Current profile configuration and case selection



Profiles are implemented. A schedule captures the saved revision and year; later profile edits do not change an existing schedule.

3. Report data and reconciliation

The shared SQL report table stores January-December values by dataset/filter scope, report year, State, RTO and Maker. Job records preserve filters, timestamps, worker assignment, status and errors. Import history records added, unchanged or conflicting values and their source references.

Rule	Data behavior and result
Source meaning	OTHERS is retained as OTHERS. Blank is different from zero. Invalid counts and contradictory duplicate rows require review.

Rule	Data behavior and result
Idempotent save	Source checksums and job/result state prevent duplicate completion/import of the same accepted result.
Conflicting values	Ordinary imports do not silently replace conflicting non-empty monthly values. Updates requiring replacement use a separately validated path.
No data	A confirmed empty source result is durable NO_DATA; it is not a timeout, rejected challenge or parsing error.
Maker reconciliation	Backend tables and GLOBAL / DISCOVER / REFRESH stages exist. The complete automatic Maker Update workflow and customer-facing orchestration are not established as complete.

All authorized users read the shared report dataset. Run completion alone does not establish complete State/RTO coverage: compare the expected profile cases with successful, no-data and unresolved outcomes.

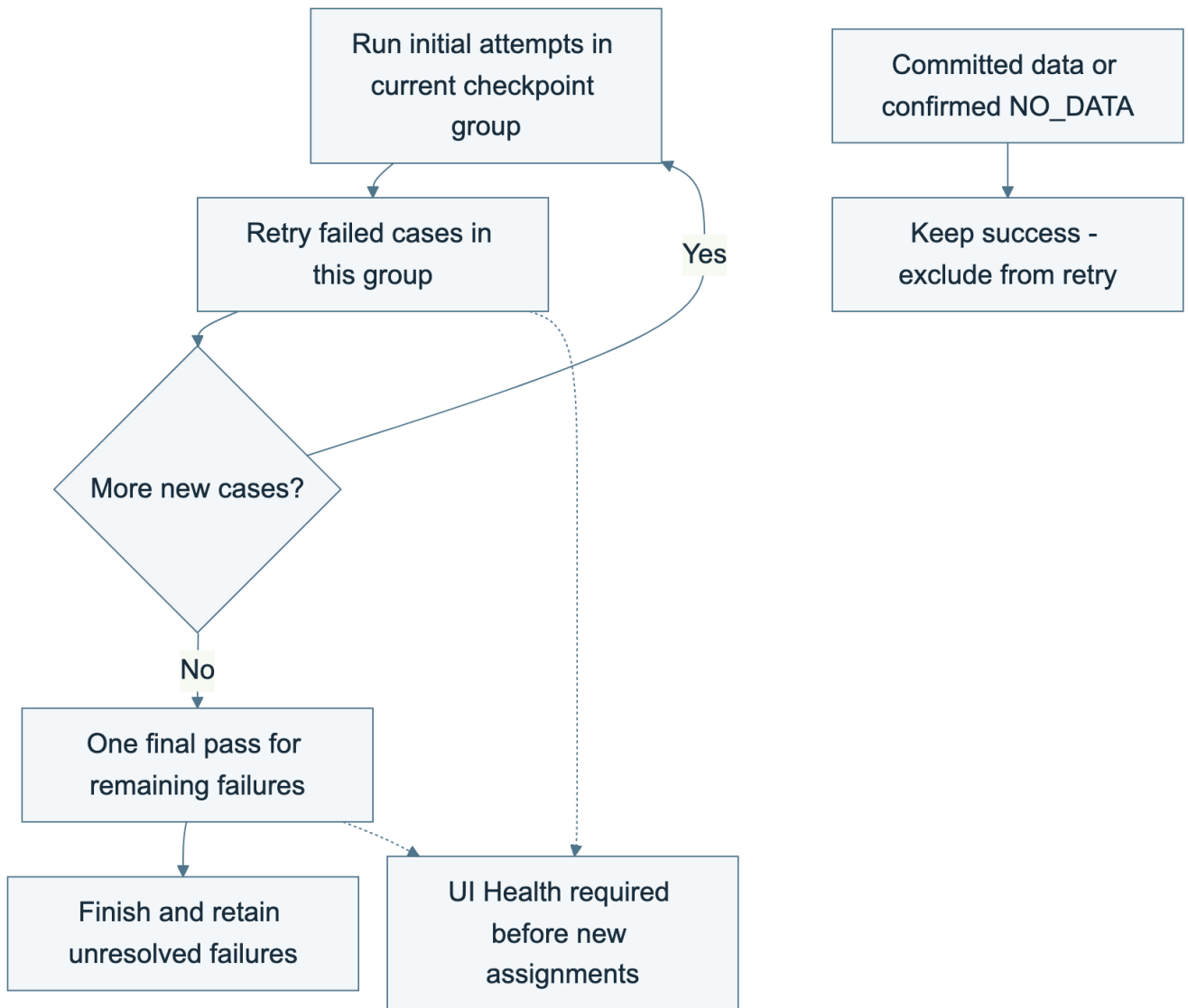
4. Operations, monitoring, and access control

The backend owns planning, dispatch and recovery; refreshing the dashboard does not start another dispatch loop. Workers pull eligible tasks through the API within the queue concurrency limit. Figure 5 shows the current retry policy.

Area	How it is supported
Continue and retry	Process the current checkpoint group, retry its failed cases, then continue to the next group. After the last group, perform one final pass for remaining failures. A case normally has up to three attempts; confirmed NO_DATA is not retried.
Pause and resume	Pause stops new claims and lets active cases save. Continue checks selected workers and UI Health again, then resumes the same SQL session/queue. Saved and no-data cases are retained; changing the limit is applied on continue.
Monitoring	Run cards show status, total/processed cases and speed. System activity exposes planning/recovery steps, worker activity and durable checkpoints; diagnostics can be copied.
Network recovery	The backend checks VAHAN connectivity. Source network loss pauses collection and returns interrupted work to the same queue without counting it as a failed attempt. Two healthy checks permit recovery through a fresh preflight. Keep paused disables automatic recovery for that schedule.
Authentication	Individual accounts, hashed passwords, authenticated downloads and admin/member permissions. Role changes and password-reset flows revoke affected sessions; the last active admin is protected.
Storage and restart	SQL stores progress, retry phase, jobs and results. Restart recovery reuses durable session checkpoints; a scheduler advisory lock selects one leader across API processes.

Delay warnings indicate stale updates or long preparation; they do not prove a hang or automatically cancel work. Manual pauses stay paused until continued. Disabling future repeats does not stop an active run; deleting an eligible schedule retains committed report data and run history.

Figure 5. Checkpoint retry and final recovery



Checkpoint groups contain at least 10 cases and can grow with concurrency. Confirmed NO_DATA is not retried.

5. Monthly collection and multiple workers: current support and planned expansion

Monthly repetition, shared queue execution and multiple worker instances now exist in the local implementation. Monthly repetition starts the saved Calendar Year report profile on the selected day, or month-end when that day is absent. It is not a separate month-only reporting mode.

Capability	Current scope / remaining expansion
Monthly schedule	Once, daily and monthly repeats use the saved time zone. Services must remain running; repeating schedules do not replay every missed occurrence.
Multiple workers	Compose can replicate the runner service. Each instance has a worker ID and isolated browser context, with one case active at a time.
Durable queue	PostgreSQL task rows are claimed through API transactions with row locking and SKIP LOCKED. No Redis queue is deployed by the current Compose file.
Future expansion	Multi-host orchestration, high availability, a separate processing service and a fully orchestrated Maker-change refresh remain separate designs requiring implementation and acceptance.

6. Delivery scope and evaluation criteria

Current source supports editable profiles, scheduled annual/month-wise report collection, PostgreSQL queue claims, configurable worker instances, workbook-to-SQL ingestion, report lookup/export, account administration, source-page preflight and bounded recovery. It does not establish acceptance of every live workflow.

Acceptance area	Required evidence
Scope and filters	Compare the saved profile revision, year, live options, previewed cases and expected/observed worker filters.
Run accounting	Reconcile planned cases with saved with-data, confirmed no-data, unresolved failures and remaining work. Check that retries do not inflate successful-case counts.
Data and Excel	Compare source worksheets, Maker labels and all 12 months against SQL and the exported workbook for the selected scope.
Scheduling and recovery	Exercise one-time/daily/monthly timing, timezone/month-end behavior, pause/continue, worker changes, API restart and network recovery without duplicate saved results.
UI Health and access	Verify every selected worker is checked, invalid contracts block new work, errors can be copied, and members cannot invoke admin functions.
Incomplete functions	Remaining-time/finish-time display, complete Maker Update orchestration and an attended verification dashboard require separate completion and validation.

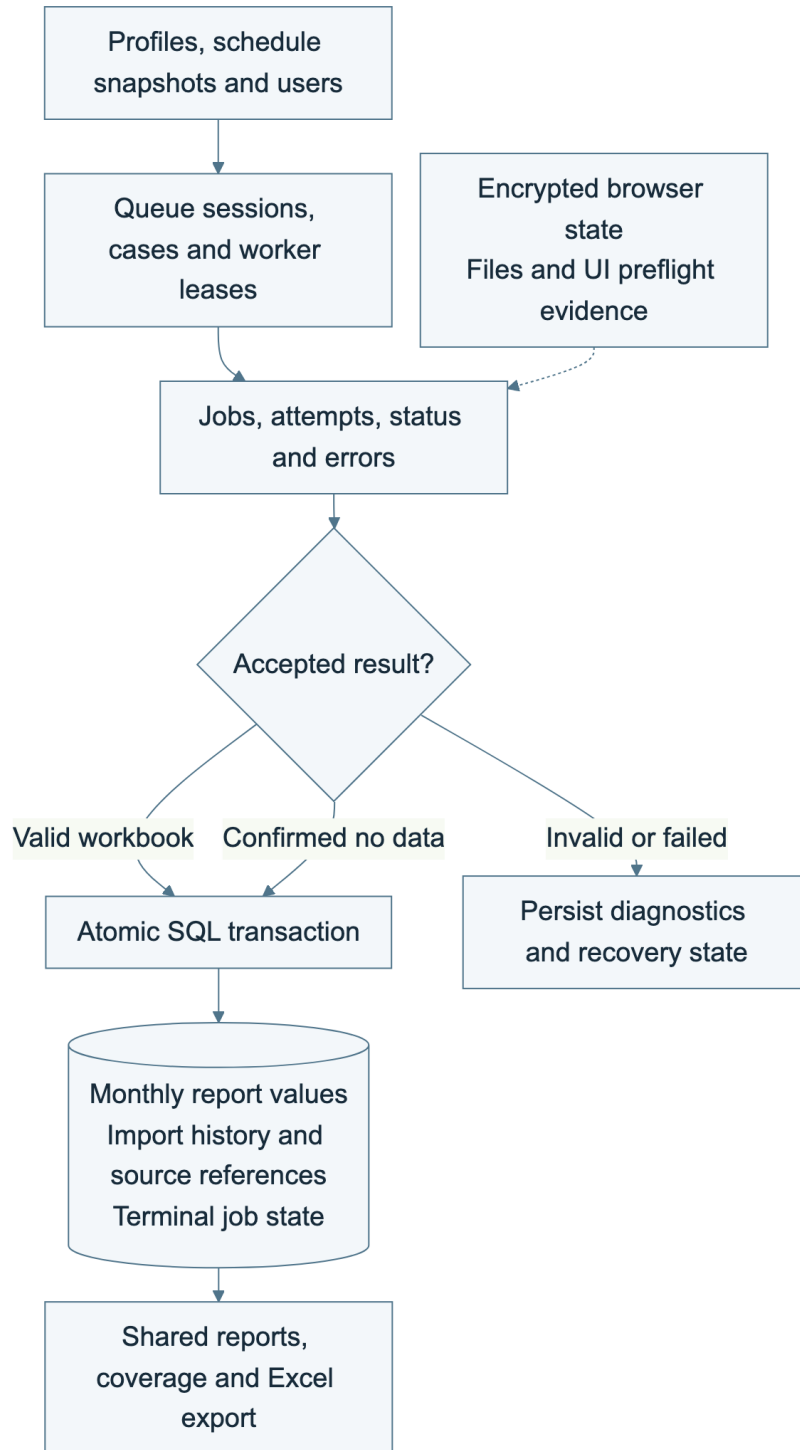
7. Database and data lifecycle

PostgreSQL is both the durable operational record and queue store. A case is successful only after its workbook or confirmed no-data result and terminal state have been committed. Figure 6 shows the lifecycle boundary.

Data group	Current function	Business result
Users and sessions	Users, auth sessions and audit events enforce access. Report sessions group cases and retain run history.	Access and activity can be traced.
Profiles and schedules	Filter profiles retain definitions/revisions. App settings retain schedules, configuration and scheduler/retry checkpoints.	Execution can resume from a saved plan.
Jobs, workers and queue	Jobs, runners, planning leases, queue sessions/tasks and events persist assignment, state, failures and progress.	One worker owns at most one active case; unfinished work remains visible.
Main report data	Maker / State / RTO / year monthly values with dataset context, source information and report update history.	Blank and zero remain distinct; exports use committed data.
Maker change data	Maker global reports, office index, update runs/tasks support partial baseline/discovery/refresh processing.	The backend supports targeted data work, while full orchestration remains incomplete.
Files and contracts	Stored files/artifacts, encrypted browser states, UI contract versions and per-worker preflight evidence have dedicated storage.	Restart state and page-check diagnostics remain auditable.

The PostgreSQL volume survives container stop/recreation. A source workbook is ingested into SQL; a second master Excel file is not required. Backup/recovery and deployment readiness must be validated separately from this documentation update.

Figure 6. Database and data lifecycle



Operational records and report values meet at the successful SQL commit boundary; the database remains the durable source for lookup and export.

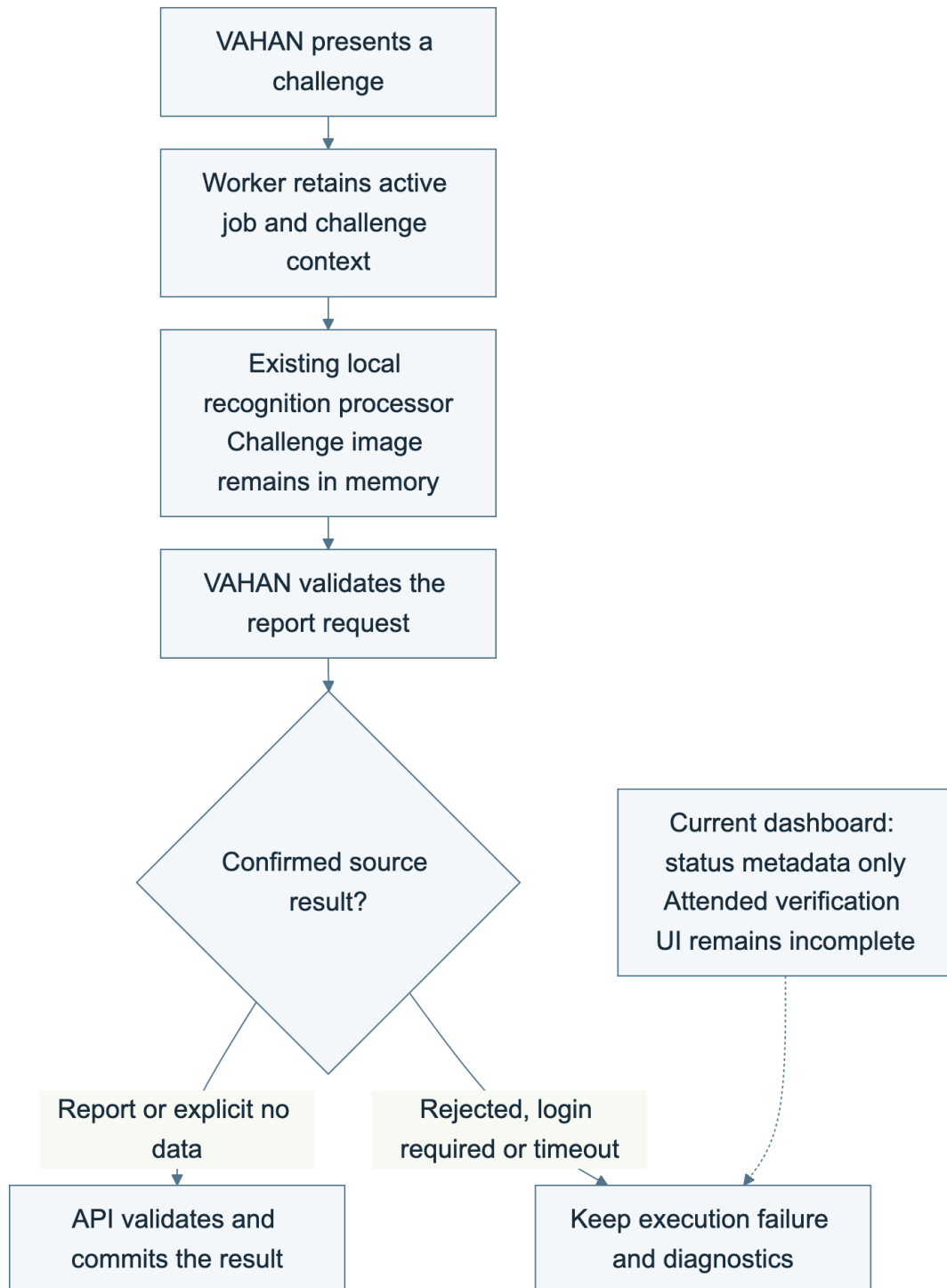
8. CAPTCHA handling and verification

The current runner contains an existing in-process Tesseract recognition path. The VAHAN challenge image stays in worker memory and is supplied to that processor through stdin. Current schedule metadata exposes job/worker state rather than a CAPTCHA image. Figure 7 describes the existing responsibility and result boundary.

Function	Current behavior	Result or control
Challenge context	The worker associates the current challenge with the active job and challenge identifier.	A changed or stale challenge must not complete another case.
Existing recognition	Local processing remains inside the worker; there is no independently deployed third-party challenge service in Compose.	Recognition availability is separate from VAHAN acceptance.
Source verification	VAHAN decides whether the submitted report request is accepted. The worker also verifies report filters and waits for an explicit result.	A recognition response alone is not a saved report.
Operator interface	The current dashboard has no challenge image, entry panel or refresh monitor. Legacy backend events are not proof of an attended UI.	An operator-assisted verification workflow remains incomplete.
Failure outcome	Rejected verification, login requirements or missing results remain diagnosable execution failures.	They must not be classified as NO_DATA or successful collection.

Recognition success and source acceptance have not been measured in this document update. Additional authentication or verification requested by VAHAN requires an explicitly supported operator workflow. The report does not add or change challenge-processing code.

Figure 7. Current verification responsibility



The existing processor runs locally in the worker. The current dashboard has no attended verification panel; VAHAN acceptance and committed data determine the result.

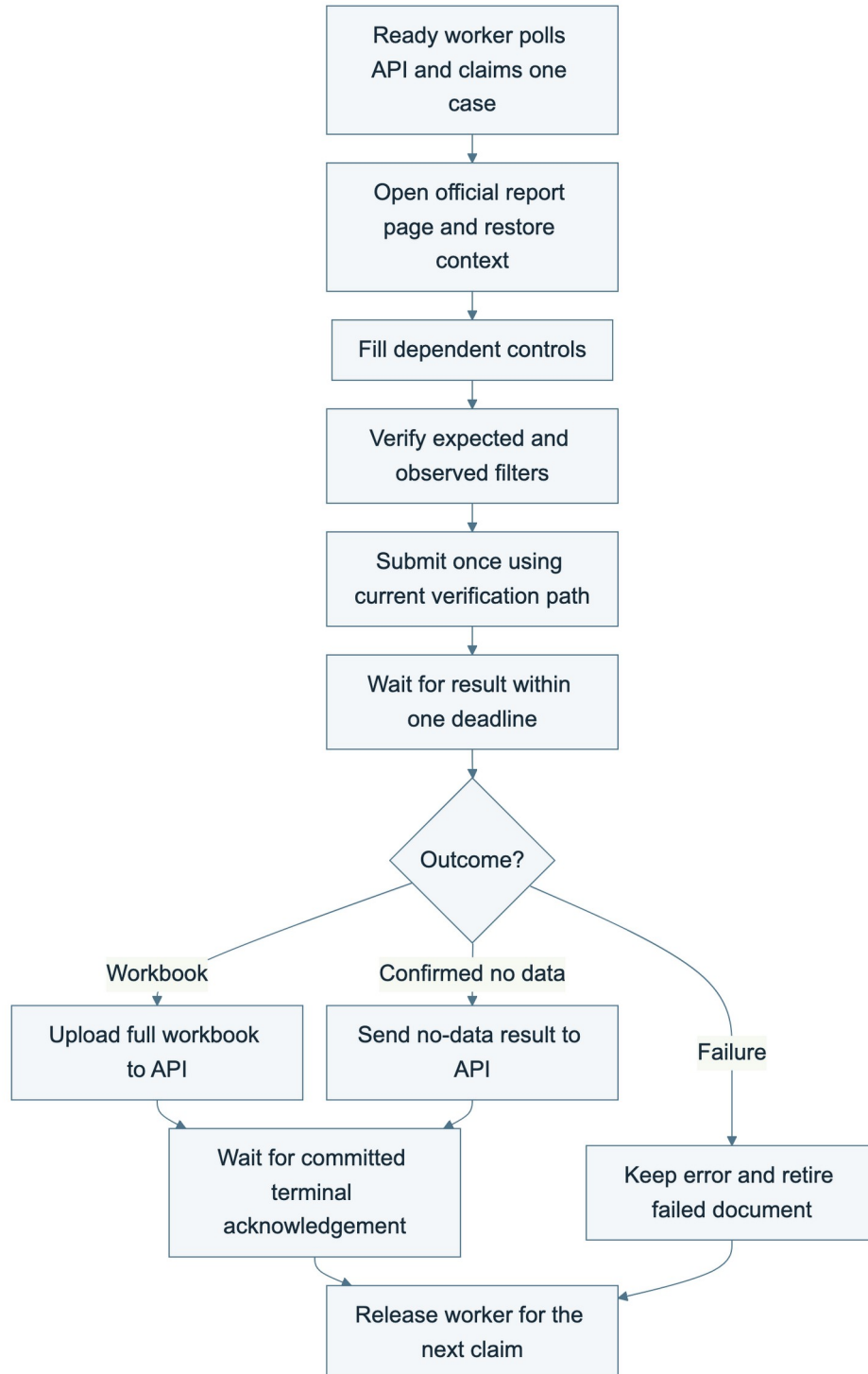
9. Crawler worker execution

Each Node.js/Playwright worker owns one Chromium browser context and processes one assigned case at a time. Ready workers poll the API and claim queued cases directly through the PostgreSQL-backed endpoint. Figure 8 shows execution stages.

Stage	Crawler function	Completion condition
Claim and prepare	Request an eligible SQL task; respect concurrency, connectivity, worker readiness, planning leases and page-check evidence; open the official report page.	The worker has a valid assignment and required source controls are available.
Fill filters	Apply dependent geography/vehicle/report controls in order; recheck expected and observed values after dynamic options load.	The API accepts the filter verification record.
Submit report	Use the current verification path, click Apply and await the report result. Reattach result reading after navigation within one deadline.	Late document navigation does not trigger a duplicate Apply.
Finish case	Download the full workbook and wait for save acknowledgement, or save confirmed no data. Retain errors and reset a failed document before the next case.	The API confirms a terminal result before the worker accepts another assignment.

Throughput depends on source latency, workbook size, browser resources and verification. Increasing the configured task limit does not create browser containers automatically or prove a proportional speed increase.

Figure 8. Crawler worker execution



A worker accepts one active case. Result reading can reattach after navigation within the same deadline without repeating Apply.

10. Excel processing and report aggregation

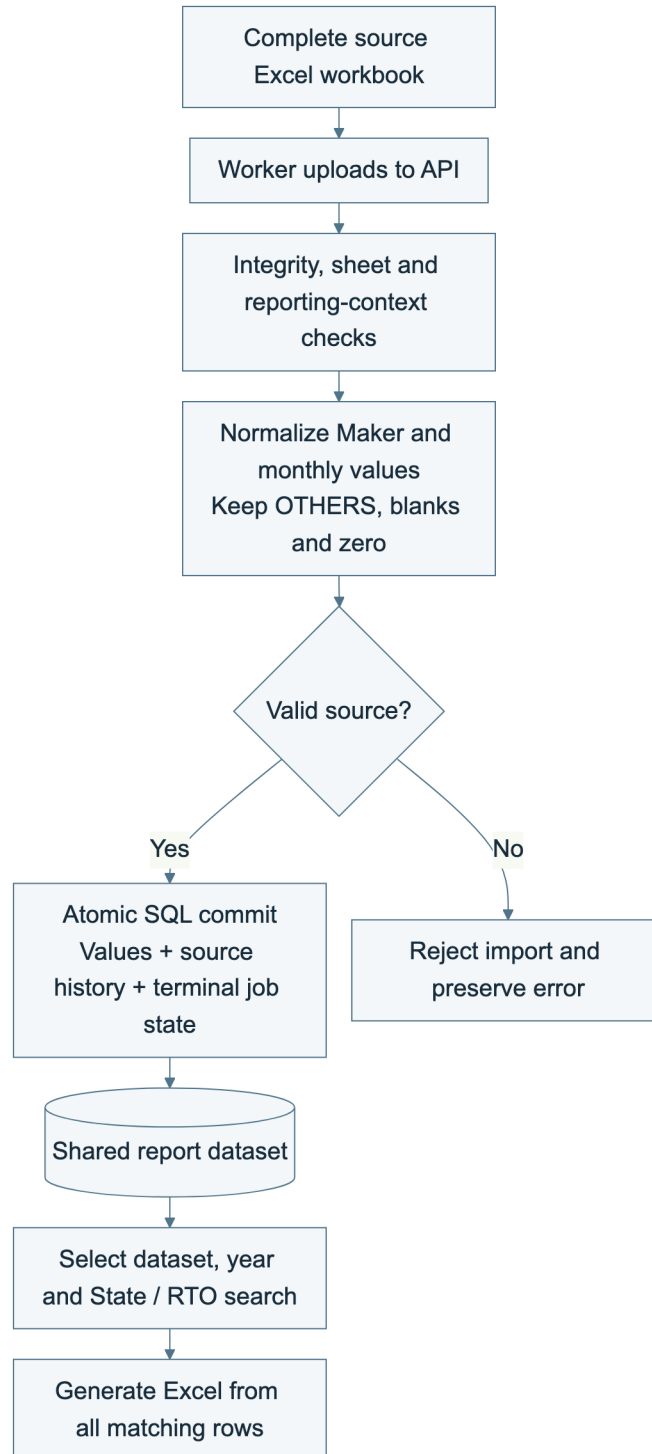
The worker downloads the complete source workbook. The API performs ingestion and SQL aggregation; there is no separate deployed Excel aggregation worker. Figure 9 shows the path from source report to export.

Stage	Current function	Data quality result
Receive source	Accept the assigned worker's completed workbook under upload-size and workbook-integrity checks.	Incomplete or damaged exports are rejected.

Stage	Current function	Data quality result
Read all sheets	Read worksheets, identify Maker/month columns and validate the reporting year and State/RTO context.	A partial or ambiguous source does not become a successful import.
Normalize values	Retain OTHERS; preserve blank versus zero; reject invalid counts and contradictory duplicates.	Monthly values retain their source meaning.
Commit results	Write accepted report values, source references, terminal job state and history within the SQL transaction boundary.	Completion is reported only after save confirmation.
Serve exports	Query the selected dataset/year and search scope; generate a new Excel workbook from committed rows, including all matching rows rather than only the current page.	Exported data reflects the saved SQL scope.
Future processing	A separate workbook processor can be introduced later while retaining validation and atomic commit semantics.	On-demand merged export remains a view of SQL data.

The worker-to-API acknowledgement is the completion boundary. A downloaded workbook that fails validation remains an error; it is not counted as no data. Deleting a completed schedule or hiding a session does not remove previously committed report values.

Figure 9. Excel processing and report aggregation



Workbook processing is performed by the API. Exports include matching committed rows, not only the visible table page.

Implementation references and review scope

Reviewed on 10 October 2026: `compose.yaml`; `README.md`; `docs/run-schedules.md`; `docs/batch-error-recovery.md`; `docs/system-status.md`; the queue repository and scheduler; schedule models; the browser runner; database schema and result repository; and current navigation, schedule/progress and coverage components.

The October 8 status document contains older observations. Where it differs from the current checkout, this revision uses the inspected source: monthly repeats and PostgreSQL task claims are present; remaining-time/finish-time labels and a complete attended CAPTCHA panel are absent from the current dashboard.

This update changes report artifacts only. It does not start jobs, edit schedules, rebuild services, apply migrations, change account records or establish live acceptance evidence.